

Software Verification and Validation

Part Time Lecturer
Talimul Bari Shreshtho

Islamic University of Technology

July 11, 2026



Table of Contents

1 White Box Testing

Definition

White-box testing is a software testing technique where the **internal structure, design, and source code** of the software are examined to design and execute test cases.

Also known as:

- Glass-box testing
- Structural testing
- Clear-box testing
- Code-based testing

Key Idea: The tester has full knowledge of the program's implementation.

Black-Box vs. White-Box Testing

Black-Box Testing	White-Box Testing
Tests external functionality	Tests internal logic and code
No source code knowledge required	Source code knowledge required
Focuses on inputs and outputs	Focuses on execution paths and statements
Usually performed by QA testers	Commonly performed by developers
Detects missing functionality	Detects logical and implementation errors

Why Do We Need White-Box Testing?

- Verifies that the implemented logic is correct.
- Detects errors hidden inside the program structure.
- Finds unreachable or dead code.
- Ensures important execution paths are tested.
- Identifies coding mistakes that functional testing may miss.
- Complements black-box testing rather than replacing it.

Remember

Black-box testing verifies **what** the software does.

White-box testing verifies **how** the software works.

Common White-Box Testing Techniques

Technique	Purpose
Statement Coverage	Execute every statement
Branch Coverage	Execute every decision outcome
Condition Coverage	Test every Boolean condition
Path Testing	Execute independent execution paths
Loop Testing	Validate loop behavior
Data Flow Testing	Track variable usage
Mutation Testing	Evaluate effectiveness of test cases

Today's focus: **Mutation Testing**

Mutation Testing

Definition

Mutation Testing is a **white-box testing technique**, primarily used during **unit testing**, to evaluate the effectiveness of a test suite.

Basic Idea

- Introduce small, intentional changes (faults) into the source code.
- Each modified version is called a **mutant**.
- Execute the existing test cases against these mutants.
- A good test suite should detect the introduced faults.

Goal

Evaluate the **quality of the test cases**, not the quality of the program.

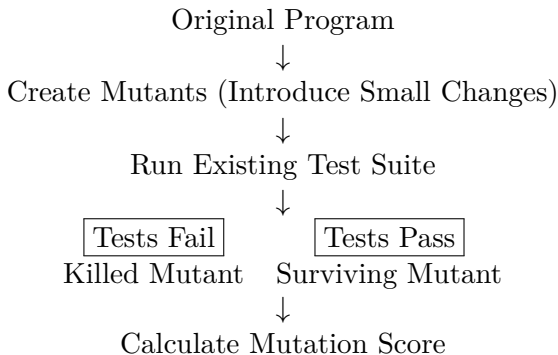
Key Concepts of Mutation Testing

- **Original Program** – The correct implementation.
- **Mutant** – A program with a small, intentional modification.
- **Killed Mutant** – A test case detects the mutation, causing the mutant to produce an incorrect result or fail.
- **Surviving Mutant** – The mutant behaves the same as the original for all existing test cases, indicating insufficient test coverage.

Objective

Design test cases with strong **fault-detection capability** so that as many mutants as possible are killed.

Mutation Testing Workflow



Primary Mutants (First-Order Mutants)

Definition

A **Primary (First-Order) Mutant** is created by applying **exactly one mutation operator** to the original program.

Original Code

```
if (a > b)
    max = a;
else
    max = b;
```

Primary Mutant

```
if (a >= b)
    max = a;
else
    max = b;
```

Mutation Operator Used: Relational Operator Replacement ($> \rightarrow \geq$)

Example: Primary (First-Order) Mutants

Original Program

```
int example(int x, int y)
{
    if (x > y)
        x = x + y;
    else
        x = y;

    return x;
}
```

Generated Mutants

- **M1:**
x = x - y;
- **M2:**
x = x / y;
- **M3:**
x = x + 1;
- **M4:**
return y;

Killing the Mutants

Test Case	(x, y)	Original	Mutant	Killed
TD ₁	(2,2)	2	0 (M1)	M1
TD ₂	(4,3)	7	1 (M2)	M2
TD ₃	(3,2)	5	4 (M3)	M3
TD ₄	(5,2)	7	2 (M4)	M4

Observation

Each test case produces a different output for the mutant than for the original program. Therefore, each mutant is **killed**.

Secondary Mutants (Higher-Order Mutants)

Definition

A **Secondary (Higher-Order) Mutant** is created by applying **two or more mutation operators** to the original program.

Original Code

```
int example(int x, int y)
{
    if (x > y)
        x = x + y;
    else
        x = y;

    return x;
}
```

Secondary Mutant

```
int example(int x, int y)
{
    if (x >= y)
        // Mutation 1
        x = x - y;
    // Mutation 2
    else
        x = y;
    return y;
    // Mutation 3
}
```

Common Types of Mutation Testing

Mutation testing introduces small changes into a program to evaluate the effectiveness of test cases.

Mutation Type	Description
Value Mutation	Changes the value of a constant or variable (e.g., increment/decrement by 1 or replace a constant).
Statement Mutation	Deletes a statement or replaces it with another valid statement.
Decision Mutation	Modifies arithmetic, relational, or logical operators used in expressions or conditions.

Value Mutation

Definition

A **Value Mutation** changes the value of a constant or variable, typically by incrementing, decrementing, or replacing it with another value.

Original Code

```
int maxIndex(int a[])
{
    int r = 0;
    for (int i = 1; i < 4; i++)
    {
        if (a[i] > a[r])
            r = i;
    }
    return r;
}
```

Value Mutant

```
int maxIndex(int a[])
{
    int r = 0;
    for (int i = 2; i < 4; i++)
    {
        if (a[i] > a[r])
            r = i;
    }
    return r;
}
```

Statement Mutation

Definition

A **Statement Mutation** deletes or replaces an executable statement to determine whether the test suite can detect the resulting fault.

Original Code

```
int maxIndex(int a[])
{
    int r = 0;
    for (int i = 1; i < 4; i++)
    {
        if (a[i] > a[r])
            r = i;
    }
    return r;
}
```

Statement Mutant

```
int maxIndex(int a[])
{
    int r = 0;
    for (int i = 1; i < 4; i++)
    {
        /* Deleted */
    }
    return r;
}
```

Definition

A **Decision Mutation** modifies the operators used in decision-making or expressions, such as **relational**, **logical**, or **arithmetic** operators, to verify whether the test suite detects the change.

Original Code

```
int max(int a, int b)
{
    if (a > b)
        return a;
    else
        return b;
}
```

Decision Mutant

```
int max(int a, int b)
{
    if (a >= b)
        return a;
    else
        return b;
}
```

Example

```
function mutationTestingExample(arr) {  
  r = 1;  
  for (i = 2; i<=3; ++i) {  
    if (a[i] > a[r]) r = i;  
  }  
}
```

```
function M4(arr) {  
  r = 1;  
  for (i = 2; i<=3; ++i) {  
    if (a[r] > a[r]) r = i;  
  }  
}
```

```
function M1(arr) {  
  r = 1;  
  for (i = 1; i<=3; ++i) {  
    if (a[i] > a[r]) r = i;  
  }  
}
```

```
function M2(arr) {  
  r = 1;  
  for (i = 2; i<=3; ++i) {  
    if (i > a[r]) r = i;  
  }  
}
```

```
function M3(arr) {  
  r = 1;  
  for (i = 2; i<=3; ++i) {  
    if (a[i] >= a[r]) r = i;  
  }  
}
```

Mutation Testing Example: Test Data

The following test cases will be used to evaluate the original program and its mutants.

Test Data	arr[1]	arr[2]	arr[3]
TD ₁	1	2	3
TD ₂	1	2	1
TD ₃	3	1	2

Purpose

These test cases will be executed on both the **original program** and its **mutants** to determine which mutants are **killed** and which **survive**.

Mutation Testing Example: Test Results

Test Data	Original	M1	M2	M3	M4
TD ₁	3	3	3	3	1
TD ₂	2	2	3	2	1
TD ₃	1	1	2	1	1

Killed Mutants

Test Data	Killed Mutants
TD ₁	M4
TD ₂	M2, M4
TD ₃	None

Observation

- **M1 and M3** are killed.
- **M2** survives, indicating that the current test suite is not sufficient to detect this mutation.

Types of Mutants

After executing a test suite, mutants can be classified into three categories:

- ❶ **Killed Mutants**
- ❷ **Survived Mutants**
- ❸ **Equivalent Mutants**

Purpose

This classification helps evaluate the effectiveness of the test suite and identify areas where additional test cases are needed.

Definition

A mutant is said to be **killed** if at least one test case produces a different output from the original program.

Characteristics

- Indicates that the test case successfully detected the injected fault.
- Demonstrates good fault-detection capability.
- Contributes positively to the mutation score.

Example

Original Output = 10

Mutant Output = 8

⇒ **Mutant is Killed.**

Definition

A mutant **survives** if all test cases produce the same output as the original program.

Characteristics

- Existing test cases fail to detect the mutation.
- Suggests insufficient test coverage.
- Additional or stronger test cases may be required.

Example

Original Output = 10

Mutant Output = 10

⇒ **Mutant Survives.**

Equivalent Mutants

Definition

An **equivalent mutant** is syntactically different from the original program but produces the same output for **every** possible input.

Characteristics

- Cannot be killed by any test case.
- Does not represent a real fault.
- Excluded when calculating the mutation score.

Example

- Original: `if (x > y)`
- Mutant: `if (y < x)`

Although written differently, both conditions are logically equivalent.

Performance Metric: Mutation Score

Mutation Score

The effectiveness of a test suite in mutation testing is measured using the **Mutation Score (MS)**.

$$\text{Mutation Score} = \frac{\text{Number of Killed Mutants}}{\text{Total Mutants} - \text{Equivalent Mutants}} \times 100\%$$

Where:

- **Killed Mutants:** Mutants detected by at least one test case.
- **Total Mutants:** Total number of generated mutants.
- **Equivalent Mutants:** Mutants that cannot be killed because they are functionally identical to the original program.

Interpretation

A higher mutation score indicates a more effective test suite with stronger fault-detection capability.

Example: Calculating Mutation Score

Suppose a program generates:

- Total Mutants = 50
- Equivalent Mutants = 5
- Killed Mutants = 40

$$\text{Mutation Score} = \frac{40}{50 - 5} \times 100 = \frac{40}{45} \times 100 = 88.89\%$$

Result

The test suite kills nearly 89% of all non-equivalent mutants, indicating good fault-detection capability.

Mutation Testing Example

Scenario

A hospital management system assigns a doctor based on the patient's age.

- If the patient's age is greater than 14, assign a General Physician.
- Otherwise, assign a Pediatrician.

Original Code

```
void routePatient(Patient patient)
{
    if (patient.age > 14)
        doctor = routeGeneralPhysician(patient);
    else
        doctor = routePediatrician(patient);
}
```

Example: Hospital

```
function M1(patient)
{
    if(patient.age < 14)
    |   doctor = routeGeneralPhysician(patient);
}
```

```
function M2(patient)
{
    if(patient.age = 14)
    |   doctor = routeGeneralPhysician(patient);
}
```

```
function M3(patient)
{
    if(patient.age >= 14)
    |   doctor = routeGeneralPhysician(patient);
}
```

```
function M4(patient)
{
    if(patient.age <= 14)
    |   doctor = routeGeneralPhysician(patient);
}
```

Example: Hospital

```
function M5(patient)
{
    if(patient.age > 14) ;
}
```

```
function M6(patient)
{
    if(patient.age > 14)
        doctor = "Dr. H";
}
```

```
function M7(patient)
{
    if(patient.age %% 14)
        doctor = routeGeneralPhysician(patient);
}
```

```
function M8(patient)
{
    if(patient.age > 14 & patient.age > 14)
        doctor = routeGeneralPhysician(patient);
}
```

Example: Hospital

Test set-data	Expected result	Mutant 1	Mutant 2	Mutant 3	Mutant 4	Mutant 5	Mutant 6	Mutant 7	Mutant 8
14	GP not assigned	Success-Not assigned	Fails- GP assigned	Fails- GP assigned	Fails- GP assigned	Nothing happens	Success-Not assigned	Syntax error	Success- GP Not assigned
15	GP is assigned	Fails- GP not assigned	Fails- GP not assigned	Success- GP assigned	Fails- GP not assigned	Nothing happens	Fails- GP not assigned	Syntax error	Success- GP assigned
0	GP not assigned	Fails- GP assigned	Success-Not assigned	Success-Not assigned	Fails- GP assigned	Nothing happens	Success-Not assigned	Syntax error	Success-Not assigned
13	GP not assigned	Fails- GP assigned	Success-Not assigned	Success-Not assigned	Fails- GP assigned	Nothing happens	Success-Not assigned	Syntax error	Success-Not assigned

Mutants to Avoid

Not all generated mutants are useful. The following types should generally be avoided:

1. Still-Born (Syntactically Incorrect) Mutants

- Mutants that contain syntax or compilation errors.
- Only **syntactically valid** mutants should be generated.

2. Equivalent Mutants

- Functionally identical to the original program.
- Produce the same output for every possible input and Cannot be killed by any test case.

3. Trivial Mutants

- Easily detected , provide no information about test quality.
- Do not help improve the effectiveness of the test suite.

Are Mutants Realistic?

Research Question

"Can artificially generated mutants accurately represent real software faults?"

Research Findings

Mutation as an Effective Testing Tool

"Our analysis suggests that mutants, when using carefully selected mutation operators and after removing equivalent mutants, can provide a good indication of the fault detection ability of a test suite."

Implications

- Carefully designed mutants resemble many real programming mistakes.
- Mutation testing is a reliable way to evaluate test suite quality.
- Removing equivalent mutants improves the accuracy of the evaluation.

Are Mutants Valid Substitutes for Real Faults?

Research Question

"Can mutants effectively represent real software faults when evaluating test suites?"

Research Findings

Evidence from Empirical Studies

Mutant detection is **positively correlated** with the detection of real software faults, regardless of the code coverage achieved by the test suite.

- Test suites that kill more mutants generally detect more real defects.
- This relationship is **independent of statement/code coverage**.
- The correlation between **mutation score and real fault detection** is stronger than the correlation between **statement coverage and real fault detection**.

Reference: Just, R., Jalali, D., Inozemtseva, L., Ernst, M. D., Holmes, R., & Fraser, G. (2014). *Are Mutants a Valid Substitute for Real Faults in Software*

Cost of Mutation Testing

Example

Assume:

- 300 functions in the codebase
- 10 test cases per function
- Each test case takes **0.2 seconds** to execute

Normal Test Execution

$$300 \times 10 \times 0.2 = 600 \text{ seconds}$$

$$600 \text{ seconds} = 10 \text{ minutes}$$

Mutation Testing

Assume each function generates **20 mutants**.

$$300 \times 20 \times 10 \times 0.2 = 12000 \text{ seconds}$$

$$12000 \text{ seconds} = 200 \text{ minutes} = 3 \text{ hours } 20 \text{ minutes}$$

Optimizing Mutation Testing

Mutation testing can be expensive, but several heuristics help reduce its execution cost.

Observation

A mutant **cannot be killed** if the test suite does not execute the statement where the mutation was introduced.

This is known as the *Reachability Condition*.

Optimization Heuristics

① Coverage-Based Test Selection

- Execute only the test cases that reach (cover) the mutated statement.
- Skip all unrelated test cases.

② Early Termination

- Once a mutant is killed by a test case, stop executing the remaining tests for that mutant.
- Additional executions provide no extra benefit.

Benefit

Optimizing Mutation Testing (Cont.)

Observation

Mutants generated using the **same mutation operator** at the **same program location** often represent similar types of faults.

Optimization Heuristic: Mutant Sampling

- Execute only a **subset of the generated mutants** instead of all mutants.
- Reduces the computational cost of mutation testing.
- The simplest and most widely used approach is **Random Sampling**.

Random Sampling

- Randomly select a percentage of mutants (e.g., 10%–30%).
- Perform mutation analysis only on the selected mutants.
- Provides a good trade-off between **execution cost** and **testing effectiveness**.

Popular Mutation Testing Tools

Several mutation testing frameworks are available for different programming languages.

Programming Language	Mutation Testing Tools
Java	PIT (PITest) MuJava Bacterio ¹ Javalanche Major Descartes
PHP	Humbug Infection PHP
JavaScript	Stryker

Most Popular Tools Today

- **Java:** PIT (PITest)



J. H. Andrews, L. C. Briand, and Y. Labiche, “Is Mutation an Appropriate Tool for Testing Experiments?” in *Proceedings of the 27th International Conference on Software Engineering (ICSE)*, 2005, pp. 402–411.



R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, “Are Mutants a Valid Substitute for Real Faults in Software Testing?” in *Proceedings of the 22nd ACM SIGSOFT International Symposium on the Foundations of Software Engineering (FSE)*, 2014, pp. 654–665.

Thank You

Thank you for your attention!

`talimulbari@iut-dhaka.edu`